

TER : Découverte et implémentation de systèmes de preuves 0-knowledge.

Raphael LEONARDI

May 2, 2025

1 Introduction

J'effectue un TER au sein de l'INRIA, dans l'équipe GRACE, dans le but de découvrir et d'implémenter des systèmes de preuves 0-knowledge. Pour cela, j'utilise le langage C et plus précisément la bibliothèque PARI GP.

La Zero Knowledge Proof ou preuve à divulgation nulle de connaissance est un protocole permettant à un utilisateur de prouver qu'une situation est réelle sans avoir à révéler d'information relative à cette dernière. Il s'agirait par exemple pour une personne de prouver son identité sans avoir à la révéler.

Les protocoles de preuve 0-Knowledge se base sur des interactions entre deux acteurs : le prouveur et le vérifieur. Le prouveur va essayer de convaincre le vérifieur qu'il connaît les informations, sans les lui révéler au début, et le vérifieur va chercher à vérifier l'honnêteté du prouveur.

2 Commitments

Dans ce qui suit, un adversaire PPT nommé $\mathcal{A}$ est une Machine de Turing interactive qui tourne en temps polynomial pour le paramètre de sécurité λ .

Definition 2.1 (Logarithme Discret).

$$P \left[\begin{array}{l} \mathbb{G} = \text{Setup}(1^\lambda, g_1, \dots, g_n) \xleftarrow{\$} \mathbb{G}; \\ a_1, \dots, a_n \in \mathbb{Z}_p \rightarrow \mathcal{A}(G, g_1, \dots, g_n) : \exists a_i \neq 0 \wedge \prod_{i=1}^n g_i^{a_i} = 1 \end{array} \right] \leq \mu(\lambda)$$

Definition 2.2 (Commitment). Un schéma d'engagement non interactif consiste en une paire d'algorithmes probabilistes à temps polynomial (Setup, Com). L'algorithme de configuration $pp \leftarrow \text{Setup}(1^\lambda)$ génère les paramètres publics pp pour le schéma, pour un paramètre de sécurité λ . L'algorithme d'engagement Com_{pp} définit une fonction $\mathcal{M}_{pp} \times \mathcal{R}_{pp} \rightarrow \mathcal{C}_{pp}$ pour l'espace de messages $\mathcal{M}_{pp}$, l'espace aléatoire $\mathcal{R}_{pp}$ et l'espace d'engagement $\mathcal{C}_{pp}$ déterminés par pp . Pour un message $x \in \mathcal{M}_{pp}$, l'algorithme tire $r \xleftarrow{\$} \mathcal{R}_{pp}$ uniformément au hasard, et calcule l'engagement $\text{com} = \text{Com}_{pp}(x; r)$.

Definition 2.3 (Engagements Homomorphiques). Un schéma d'engagement homomorphique est un schéma d'engagement non interactif tel que $\mathcal{M}_{pp}$, $\mathcal{R}_{pp}$ et $\mathcal{C}_{pp}$ sont tous des groupes abéliens, et pour tous $x_1, x_2 \in \mathcal{M}_{pp}$, $r_1, r_2 \in \mathcal{R}_{pp}$, nous avons :

$$\text{Com}(x_1; r_1) + \text{Com}(x_2; r_2) = \text{Com}(x_1 + x_2; r_1 + r_2)$$

Definition 2.4 (Commitment de Pederson de vecteur). $\mathcal{M}_{pp} = \mathbb{Z}_p^n$, $\mathcal{R}_{pp} = \mathbb{Z}_p$, $\mathcal{C}_{pp} = \mathbb{G}$ d'ordre p

Setup: $g = (g_1, \dots, g_n)$, $h \xleftarrow{\$} \mathbb{G}$

Com($x=(x_1, \dots, x_n)$; r) = $h^r g^x = h^r \prod_i g_i^{x_i} \in \mathbb{G}$

Definition 2.5 (Engagement de dissimulation). Un schéma d'engagement est dit *masquant* si pour tout adversaire $\mathcal{A}$, il existe une fonction négligeable $\mu(\lambda)$ telle que :

$$P \left[b = b' \left| \begin{array}{l} p \leftarrow \text{Setup}(1^\lambda); \\ (x_0, x_1) \in M_{pp}^2 \leftarrow \mathcal{A}(pp); br \xleftarrow{\$} \{0, 1\}, r \xleftarrow{\$} R_{pp}, \\ com = \text{Com}(x_b; r); b' \leftarrow \mathcal{A}(pp, com); \end{array} \right. - \frac{1}{2} \right] \leq \mu(\lambda)$$

où la probabilité est prise sur b, r, Setup et $\mathcal{A}$. Si $\mu(\lambda) = 0$, on dit que le schéma est parfaitement masquant.

Definition 2.6 (Engagement de liage). Un schéma d'engagement est dit *liant* si pour tout adversaire $\mathcal{A}$, il existe une fonction négligeable $\mu(\lambda)$ telle que :

$$P \left[\text{Com}(x_0; r_0 = \text{Com}(x_1; r_1)) \wedge x_0 \neq x_1 \left| \begin{array}{l} pp \leftarrow \text{Setup}(1^\lambda), \\ x_0, x_1, r_0, r_1 \leftarrow \mathcal{A}(pp) \end{array} \right. \right] \leq \mu(\lambda)$$

où la probabilité est prise sur Setup et $\mathcal{A}$. Si $\mu(\lambda) = 0$, on dit que le schéma est parfaitement liant.

Dans ce qui suit, l'ordre p des groupes utilisés dépend implicitement du paramètre de sécurité λ afin de garantir que le logarithme discret dans ces groupes soit intractable pour les adversaires PPT.

3 Implémentation en C d'un groupe

On aura besoin au cours de ce projet d'utiliser des groupes en C, en utilisant la bibliothèque Pari GP. Voici comment j'ai défini ce qu'était un groupe en C :

```
1 typedef enum {
2     ADD = 0,
3     MUL = 1
4 } loiComp;
5
6
7 // On definit ce qu est une loi de groupe
8 // C est a dire une fonction prennant GEN x, GEN y
9 // Et renvoyant une loi de groupe dessus ()
10 // Une loi est une fonction de signature (GEN, GEN) -> GEN
11 typedef GEN (*Loi)(GEN, GEN);
12
13 typedef struct{
14     Loi mul;
15     GEN one;
16 } Group;
```

On a créé un typé énuméré pour la loi de composition pour utiliser des groupes tels que $(\mathbb{Z}/n\mathbb{Z})$.

4 Première Implémentation d'un algorithme sur des groupes

Pour commencer j'ai effectué une première implémentation de preuves en me basant sur la méthode décrite dans l'article page 16. Bulletproofs: Short Proofs for Confidential Transactions and More J'ai

choisi d'implémenter le prouveur et le vérifieur comme étant des structures en C, comme définis ici dans ce fichier .h :

```
1 typedef enum {
2     REJECT = 0,
3     ACCEPT = 1
4 }
```

```

5 } result;
6
7 typedef struct{
8     GEN g;
9     GEN h;
10    GEN u;
11    GEN P;
12    GEN a;
13    GEN b;
14
15    Group *groupe;
16
17 } prover;
18
19 typedef struct{
20     GEN g;
21     GEN h;
22     GEN u;
23     GEN P;
24     Group *groupe;
25
26
27 } verifieur;

```

4.1 But de l'Algorithme

L'algorithme fonctionne sur une structure de groupe G d'on on connaît des générateurs g, h (g et h contiennent chacune des générateurs). Ces générateurs g et h sont les entrées de l'algorithme, ainsi que $P \in G$ et c un scalaire $\in (Z/pZ)$ avec p premier.

Le but du prouveur sera de convaincre le vérifieur qu'il connaît deux vecteurs $a, b \in Z^n$ tels que :

$$P = g^a h^b \quad \text{et} \quad c = \langle a, b \rangle$$

Pour cela, on va introduire un élément $u \in G$. On va calculer $c = \langle a, b \rangle$. Pour vérifier que le produit scalaire est validé, on va plutôt vérifier que :

$$P = g^a h^b u^c$$

4.2 Entrées et sorties de l'algorithme

Soit p un nombre premier, et $n = 2^k, k \in N$.

Entrées : $g, h \in G^n, u, P \in G, a, b \in \mathbb{Z}_p^n$

Entrée prouveur $P : g, h, a, b, u, P$

Entrée vérifieur $V : g, h, u, P$

Sortie : ACCEPT ou REJECT

4.3 Etapes de l'algorithme

Voici l'algorithme tel qu'il est défini dans l'article :

input: $(\mathbf{g}, \mathbf{h} \in \mathbb{G}^n, u, P \in \mathbb{G} ; \mathbf{a}, \mathbf{b} \in \mathbb{Z}_p^n)$	(10)
$\mathcal{P}_{\text{IP}}$'s input: $(\mathbf{g}, \mathbf{h}, u, P, \mathbf{a}, \mathbf{b})$	(11)
$\mathcal{V}_{\text{IP}}$'s input: $(\mathbf{g}, \mathbf{h}, u, P)$	(12)
output: $\{\mathcal{V}_{\text{IP}}$ accepts or $\mathcal{V}_{\text{IP}}$ rejects	(13)
if $n = 1$:	(14)
$\mathcal{P}_{\text{IP}} \rightarrow \mathcal{V}_{\text{IP}} : a, b \in \mathbb{Z}_p$	(15)
$\mathcal{V}_{\text{IP}}$ computes $c = a \cdot b$ and checks if $P = g^a h^b u^c$:	(16)
if yes, $\mathcal{V}_{\text{IP}}$ accepts; otherwise it rejects	(17)
else: $(n > 1)$	(18)
$\mathcal{P}_{\text{IP}}$ computes:	(19)
$n' = \frac{n}{2}$	(20)
$c_L = \langle \mathbf{a}_{[:n']}, \mathbf{b}_{[n':]} \rangle \in \mathbb{Z}_p$	(21)
$c_R = \langle \mathbf{a}_{[n':]}, \mathbf{b}_{[:n']} \rangle \in \mathbb{Z}_p$	(22)
$L = \mathbf{g}_{[n':]}^{\mathbf{a}_{[:n']}} \mathbf{h}_{[:n']}^{\mathbf{b}_{[n':]}} u^{c_L} \in \mathbb{G}$	(23)
$R = \mathbf{g}_{[:n']}^{\mathbf{a}_{[n':]}} \mathbf{h}_{[n':]}^{\mathbf{b}_{[:n']}} u^{c_R} \in \mathbb{G}$	(24)
$\mathcal{P}_{\text{IP}} \rightarrow \mathcal{V}_{\text{IP}} : L, R$	(25)
$\mathcal{V}_{\text{IP}} : x \xleftarrow{\$} \mathbb{Z}_p^*$	(26)
$\mathcal{V}_{\text{IP}} \rightarrow \mathcal{P}_{\text{IP}} : x$	(27)
$\mathcal{P}_{\text{IP}}$ and $\mathcal{V}_{\text{IP}}$ compute:	(28)
$\mathbf{g}' = \mathbf{g}_{[:n']}^{x^{-1}} \circ \mathbf{g}_{[n':]}^x \in \mathbb{G}^{n'}$	(29)
$\mathbf{h}' = \mathbf{h}_{[:n']}^x \circ \mathbf{h}_{[n':]}^{x^{-1}} \in \mathbb{G}^{n'}$	(30)
$P' = L^{x^2} P R^{x^{-2}} \in \mathbb{G}$	(31)
$\mathcal{P}_{\text{IP}}$ computes:	(32)
$\mathbf{a}' = \mathbf{a}_{[:n']} \cdot x + \mathbf{a}_{[n':]} \cdot x^{-1} \in \mathbb{Z}_p^{n'}$	(33)
$\mathbf{b}' = \mathbf{b}_{[:n']} \cdot x^{-1} + \mathbf{b}_{[n':]} \cdot x \in \mathbb{Z}_p^{n'}$	(34)

4.4 Implémentation de l'algorithme en langage C :

L'algorithme est récursif, à chaque itération on divise la taille des paramètres g, h par 2, jusqu'à arriver à 1, au cas de base. On doit faire la vérification dans ce cas base, voila comment j'ai décidé de l'implémenter en C :

```

1  /*La partie finale de la preuve*/
2  result check(verifieur *v, GEN a, GEN b, GEN g, GEN h, GEN P){
3      /*Calcule dans un premier temps c*/
4
5
6
7      GEN c = dot(a, b);
8
9      Loi mul = v->groupe->mul;
10
11
12     /*On calcule g^a * h^b * u^c */
13     GEN gauche = powerVector(v->groupe, g, a);
14
15     GEN droite = powerVector(v->groupe, h, b);

```

```

16  GEN fin = power(v->groupe, v->u, c);
17
18  GEN prod = mul(gauche, mul(droite, fin));
19  /*On verifie que prod egal a P*/
20
21  // intResult transforme un entier en ACCEPTED ou REJECTED
22  return intResult(gequal(prod, P));
23
24  }

```

Quand à la partie récursive, on doit recalculer des éléments g, h comme ceci :

```

1  GEN leftRight = proverfirstComputation(bob, a, b, g, h);
2  GEN L = gel(leftRight, 1); GEN R = gel(leftRight, 2);
3
4  GEN x = verifierChooseX(p);
5  GEN common = computationCommon(bob->groupe, L, R, P, g, h, x);
6  GEN g2 = gel(common, 1); GEN h2 = gel(common, 2); GEN P2 = gel(common, 3);
7
8  GEN ab = proversecondComputation(bob, a, b, x);
9
10 GEN a2 = gel(ab, 1); GEN b2 = gel(ab, 2);
11
12 return protocolRecursive(bob, alice, g2, h2, u, P2, a2, b2, n/2, p);

```

On a utilisé plusieurs sous fonctions dans la partie récursive :

1. *proverfirstComputation()* qui calcule les éléments L et R décrits dans l'algorithme pour le prouveur uniquement.
2. *verifierChooseX()* qui prend en argument un nombre entier et renvoie un élément au hasard dans $(\mathbb{Z}/p\mathbb{Z})^*$. Attention ici p doit être premier, en effet si p ne l'était pas, l'élément x^{-1} n'aurait pas de garantie d'exister.
3. *computationCommon()* qui est faite par le vérifieur et le prouveur, et qui calcule les valeurs de g et h pour la prochaine itération.
4. *proverSecondComputation()* qui est calculée par le prouveur, et qui calcule les valeurs de a et b pour la prochaine itération.

Ensuite on lance l'appel récursif sur les nouveaux paramètres, jusqu'à atteindre le cas de base.

5 Utilisation de l'algorithme sur un exemple

Maintenant que l'on a implémenté notre algorithme en C, il faut maintenant pouvoir le tester sur des vraies valeurs.

5.1 Obtenir P et en fonction de g, h, a, b, u

Le prouveur essaie de convaincre le vérifieur qu'il connaît a, b qui vérifient l'égalité. Pour pouvoir tester notre fonction, on a besoin d'obtenir P en fonction des paramètres g, h, a, b, u , que l'on obtient par la fonction suivante en C :

```

1  GEN buildP(Group *G, GEN g, GEN h, GEN a, GEN b, GEN u){
2  GEN G2 = powerVector(G, \left( g, a);
3  GEN c = dot(a, b);
4  GEN G3 = powerVector(G, h, b);
5  GEN G4 = powerVector(G, u, c);
6
7

```

```

8   return G->mul(G2, G->mul(G3, G4));
9
10  }

```

5.2 Exemple

Maintenant que l'on a l'algorithme, et les bonnes valeurs, on peut enfin s'en servir sur un exemple. Prenons le groupe $G = (Z/pZ, +)$, avec p un nombre premier. Ici on va d'abord se concentrer sur des petits nombres pour pouvoir trouver les erreurs facilement si il y en a. Voici les valeurs que l'on prendra :

$$p = 17, G = (Z/17Z, +), n = 8, u = \overline{10}, P = \overline{12}$$

$$g = [\overline{4}, \overline{5}, \overline{7}, \overline{8}, \overline{9}, \overline{12}, \overline{13}, \overline{15}], h = [\overline{7}, \overline{2}, \overline{3}, \overline{4}, \overline{12}, \overline{1}, \overline{14}, \overline{16}], a = [\overline{4}, \overline{5}, \overline{6}, \overline{2}, \overline{1}, \overline{5}, \overline{9}, \overline{15}], b = [\overline{3}, \overline{7}, \overline{8}, \overline{16}, \overline{4}, \overline{3}, \overline{2}, \overline{7}]$$

J'ai ajouté un affichage, qui affichent les valeurs calculées par le vérifieur et le prouveur. Si l'on redirige le dans un Markodwn, on obtient un joli visuel. Voici ce que l'affichage a donnée pour cet exemple :

```

Lancement protocole avec n = 8 et p = 17
-----
## Appel recursif avec n = 8
-   a = [Mod(4, 17), Mod(5, 17), Mod(6, 17), Mod(2, 17), Mod(1, 17), Mod(5, 17), Mod(9, 17), Mod(15, 17)]
-   b = [Mod(3, 17), Mod(7, 17), Mod(8, 17), Mod(16, 17), Mod(4, 17), Mod(3, 17), Mod(2, 17), Mod(7, 17)]
-   g = [Mod(4, 17), Mod(5, 17), Mod(7, 17), Mod(8, 17), Mod(9, 17), Mod(12, 17), Mod(13, 17), Mod(15, 17)]
-   h = [Mod(7, 17), Mod(2, 17), Mod(3, 17), Mod(4, 17), Mod(12, 17), Mod(1, 17), Mod(14, 17), Mod(16, 17)]
Le prouveur a calcule L = Mod(9, 17) et R = Mod(9, 17)
Le verifieur a choisi x = Mod(7, 17)
Les calculs en communs on ete effectues.
Le prouveur et le verifieur ont calcule :
-   g' = [Mod(15, 17), Mod(7, 17), Mod(7, 17), Mod(9, 17)]
-   h' = [Mod(7, 17), Mod(2, 17), Mod(6, 17), Mod(6, 17)]
-   P' = Mod(15, 17)
-----
## Appel recursif avec n = 4
-   a = [Mod(16, 17), Mod(9, 17), Mod(2, 17), Mod(4, 17)]
-   b = [Mod(9, 17), Mod(5, 17), Mod(3, 17), Mod(10, 17)]
-   g = [Mod(15, 17), Mod(7, 17), Mod(7, 17), Mod(9, 17)]
-   h = [Mod(7, 17), Mod(2, 17), Mod(6, 17), Mod(6, 17)]
Le prouveur a calcule L = Mod(16, 17) et R = Mod(12, 17)
Le verifieur a choisi x = Mod(5, 17)
Les calculs en communs on ete effectues.
Le prouveur et le verifieur ont calcule :
-   g' = [Mod(4, 17), Mod(9, 17)]
-   h' = [Mod(9, 17), Mod(1, 17)]
-   P' = Mod(0, 17)
-----
## Appel recursif avec n = 2
-   a = [Mod(9, 17), Mod(5, 17)]
-   b = [Mod(10, 17), Mod(0, 17)]
-   g = [Mod(4, 17), Mod(9, 17)]
-   h = [Mod(9, 17), Mod(1, 17)]
Le prouveur a calcule L = Mod(13, 17) et R = Mod(3, 17)
Le verifieur a choisi x = Mod(12, 17)
Les calculs en communs on ete effectues.
Le prouveur et le verifieur ont calcule :
-   g' = [Mod(12, 17)]
-   h' = [Mod(16, 17)]
-   P' = Mod(13, 17)
-----
## Verification finale sur :
-   a = [Mod(5, 17)]
-   b = [Mod(15, 17)]
-   g = [Mod(12, 17)]
-   h = [Mod(16, 17)]
-   u = Mod(10, 17)
-   P = Mod(13, 17)
Verdict verifieur : ACCEPT

```

Dans cet exemple, le vérifieur a accepté les arguments du prouveur, les valeurs sont bonnes. Maintenant voici ce qui se passe si l'on change des valeurs quelconque dans a et b, qui rendront fausses les conjectures. Prenons comme nouvelles valeurs de a et b:

$$a = [\overline{1}, \overline{5}, \overline{2}, \overline{2}, \overline{1}, \overline{5}, \overline{9}, \overline{15}], b = [\overline{3}, \overline{7}, \overline{8}, \overline{16}, \overline{2}, \overline{3}, \overline{2}, \overline{7}]$$

On obtient ce résultat :

```

Lancement protocole avec n = 8 et p = 17
-----
## Appel recursif avec n = 8
- a = [Mod(1, 17), Mod(5, 17), Mod(2, 17), Mod(2, 17), Mod(1, 17), Mod(5, 17), Mod(9, 17), Mod(15, 17)]
- b = [Mod(3, 17), Mod(7, 17), Mod(8, 17), Mod(16, 17), Mod(2, 17), Mod(3, 17), Mod(2, 17), Mod(7, 17)]
- g = [Mod(4, 17), Mod(5, 17), Mod(7, 17), Mod(8, 17), Mod(9, 17), Mod(12, 17), Mod(13, 17), Mod(15, 17)]
- h = [Mod(7, 17), Mod(2, 17), Mod(3, 17), Mod(4, 17), Mod(12, 17), Mod(1, 17), Mod(14, 17), Mod(16, 17)]
Le prouveur a calcule L = Mod(2, 17) et R = Mod(9, 17)
Le verifieur a choisi x = Mod(2, 17)
Les calculs en communs on ete effectues.
Le prouveur et le verifieur ont calcule :
- g' = [Mod(3, 17), Mod(1, 17), Mod(4, 17), Mod(0, 17)]
- h' = [Mod(3, 17), Mod(13, 17), Mod(13, 17), Mod(16, 17)]
- P' = Mod(1, 17)
-----
## Appel recursif avec n = 4
- a = [Mod(11, 17), Mod(4, 17), Mod(0, 17), Mod(3, 17)]
- b = [Mod(14, 17), Mod(1, 17), Mod(8, 17), Mod(5, 17)]
- g = [Mod(3, 17), Mod(1, 17), Mod(4, 17), Mod(0, 17)]
- h = [Mod(3, 17), Mod(13, 17), Mod(13, 17), Mod(16, 17)]
Le prouveur a calcule L = Mod(6, 17) et R = Mod(10, 17)
Le verifieur a choisi x = Mod(2, 17)
Les calculs en communs on ete effectues.
Le prouveur et le verifieur ont calcule :
- g' = [Mod(1, 17), Mod(9, 17)]
- h' = [Mod(4, 17), Mod(0, 17)]
- P' = Mod(2, 17)
-----
## Appel recursif avec n = 2
- a = [Mod(5, 17), Mod(1, 17)]
- b = [Mod(6, 17), Mod(2, 17)]
- g = [Mod(1, 17), Mod(9, 17)]
- h = [Mod(4, 17), Mod(0, 17)]
Le prouveur a calcule L = Mod(0, 17) et R = Mod(10, 17)
Le verifieur a choisi x = Mod(9, 17)
Les calculs en communs on ete effectues.
Le prouveur et le verifieur ont calcule :
- g' = [Mod(15, 17)]
- h' = [Mod(2, 17)]
- P' = Mod(8, 17)
-----
## Verification finale sur :
a = [Mod(13, 17)]
b = [Mod(13, 17)]
g = [Mod(15, 17)]
h = [Mod(2, 17)]
u = Mod(10, 17)
P = Mod(8, 17)
Verdict verifieur : REJECT

```

Ici, le vérifieur a rejeté les arguments du prouveur, ce qui était attendu étant donné que l'on a changé les valeurs de a et de b , pour que les hypothèses sur h, g, a, b, u, P soient fausses.

6 Application aux courbes elliptiques

Maintenant que l'on a implémenté l'algorithme pour des (Z/pz) pour tester, on va l'implémenter pour des courbes elliptiques. On va pour cela changer de fichier main, et en créer un nommé *main2.c*

6.1 Définitions

Definition 6.1 (Introduction). Une courbe elliptique est l'ensemble des solutions (x, y) telles que:

$$y^2 = x^3 + ax + b, \quad a, b \in K$$

Ici on va travailler avec des courbes elliptiques avec des points dans $(Z/pZ)^2$, p premier.

Donc, les points appartenants à une courbe elliptique dans notre contexte seraient les solutions entières de l'équation :

$$y^2 \equiv x^3 + ax + b [p]$$

6.2 Précisions sur le cardinal de la courbe elliptique

On a choisi comme exemple une courbe ayant 90 points à titre d'exemple, on ne pourra pas s'en servir comme courbe elliptique. Le groupe G utilisé doit être isomorphe à $(Z/pZ)^*$. Cela signifie qu'ils doivent

6.5 Utilisation de la courbe

Maintenant que l'on a codé l'algorithme, que l'on s'est assuré qu'il fonctionne et que l'on a une courbe elliptique sur laquelle l'algorithme fonctionnera, il nous faut générer des paramètres pour le protocole.

Pour cela, on va utiliser quelques fonctions utilitaires pour générer les paramètres :

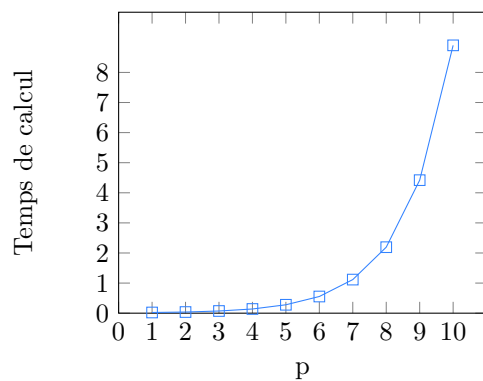
- *genPoints*(Group *G, GEN gen, int n) qui va générer n points aléatoires du groupe G en utilisant le générateur gen (chaque point étant ce générateur mis à une puissance au hasard).
- *genZpz*(GEN p, int n) qui renvoie n points aléatoirement choisis dans $(\mathbb{Z}/p\mathbb{Z})$ privé de 0.
- *initParameters*(int length, Group *G, GEN gen, GEN ordre) qui va combiner les deux fonctions et va initialiser tous les paramètres g, h, a, b, u, P nécessaires. *length* étant la taille des vecteurs, G le groupe (ici la courbe elliptique P-256) et *ordre* l'ordre du groupe G.

On peut désormais tester le protocole sur des vraies valeurs de courbe elliptique. Voici un exemple de calcul pour la courbe P-256 avec des vecteurs de taille 2^{10} (1024).

```
raphael@raphael-IdeaPad-3-15IGL05:~/Documents/TER$ ./main2 10
Simulation avec p = 10, générateurs de taille 1024
Paramètres générés en 8.743791 secondes:
Resultat : ACCEPT calculé en : 26.17330899999999714 secondes
```

Intéressons nous maintenant aux performances de l'algorithme pour des valeurs de p allant de 1 à 10 :

Génération des paramètres de taille 2^p



Protocole ZERO-KNOWLEDGE

